OpenMC unit-cell modeling: cheat sheet

What is the minimum complete model for an infinite pin lattice?

19 LESSON

Read: Condensed from the PWR and SFR unit-cell notebooks in Lessons 17 and 19.

The five pieces of an OpenMC model

Materials → surfaces/regions → cells/geometry → settings → tallies. Bundle them in `openmc.Model`, run in an isolated directory, and read the resulting statepoint. Use reflective outer and axial boundaries for a no-leakage, infinite-lattice k_∞ model. Internal fuel/cladding surfaces remain transmitting (the default).

1 Shared setup and dimensions

```python
import os, tempfile
from pathlib import Path
import numpy as np
import openmc

xs = os.environ["OPENMC_CROSS_SECTIONS"]
openmc.config["cross_sections"] = xs

rf, rc, P, H = 0.45, 0.46, 1.20, 1.0 # cm
fuel_s = openmc.ZCylinder(r=rf)
clad_s = openmc.ZCylinder(r=rc)
z0 = openmc.ZPlane(z0=-H/2, boundary_type="reflective")
z1 = openmc.ZPlane(z0= H/2, boundary_type="reflective")
axial = +z0 & -z1
```

Always check $P > 2r_c$. Use one consistent axial height H in the geometry and volumes; $H = 1$ cm is convenient.

2 PWR materials: square lattice

```python
fuel = openmc.Material(name="UO2 fuel")
fuel.add_element("U", 1.0, enrichment=4.0) # wt% U-235
fuel.add_element("O", 2.0)
fuel.set_density("g/cm3", 10.5)
fuel.temperature = 1200.0

clad = openmc.Material(name="Zr cladding")
clad.add_element("Zr", 1.0)
clad.set_density("g/cm3", 6.6)

cool = openmc.model.borated_water(
    boron_ppm=800.0, temperature=600.0,
    pressure=15.5, temp_unit="K", press_unit="MPa")
```

`borated_water` supplies H_2O composition and `c_H_in_H2O` thermal scattering. Enrichment passed to `add_element` is weight percent.

3 PWR regions, cells, and volumes

```python
x0 = openmc.XPlane(x0=-P/2, boundary_type="reflective")
x1 = openmc.XPlane(x0= P/2, boundary_type="reflective")
y0 = openmc.YPlane(y0=-P/2, boundary_type="reflective")
y1 = openmc.YPlane(y0= P/2, boundary_type="reflective")
outer = +x0 & -x1 & +y0 & -y1 & axial

cells = [
  openmc.Cell(name="fuel", fill=fuel,
              region=-fuel_s & outer),
  openmc.Cell(name="cladding", fill=clad,
              region=+fuel_s & -clad_s & outer),
  openmc.Cell(name="coolant", fill=cool,
              region=+clad_s & outer)]
V = np.array([np.pi*rf**2,
              np.pi*(rc**2-rf**2),
              P**2-np.pi*rc**2])*H
for c, v in zip(cells, V): c.volume = float(v)
```

4 SFR materials: explicit enrichment basis

```python
e = 0.15  # U-235 atom fraction among uranium atoms
fuel = openmc.Material(name="UO2 fuel")
fuel.add_nuclide("U235", e)
fuel.add_nuclide("U238", 1.0-e)
fuel.add_element("O", 2.0)
fuel.set_density("g/cm3", 10.5)

clad = openmc.Material(name="steel cladding")
clad.add_element("Fe", 1.0)       # shortened example
clad.set_density("g/cm3", 7.13)

cool = openmc.Material(name="sodium coolant")
cool.add_nuclide("Na23", 1.0)
cool.set_density("g/cm3", 0.837)
```

For a specified alloy, replace the shortened Fe material by its full elemental composition. Adding U-235 and U-238 explicitly makes e an *atom fraction*; do not confuse it with the weight-percent shortcut used in the PWR example.

5 SFR regions: hexagonal lattice

Here P is the nearest-neighbor pitch (the hexagon's across-flats dimension), so the hexagon side length is $P/\sqrt{3}$.

```python
hex_s = openmc.model.HexagonalPrism(
    edge_length=P/np.sqrt(3.0), orientation="x",
    boundary_type="reflective")
outer = -hex_s & axial

cells = [
  openmc.Cell(name="fuel", fill=fuel,
              region=-fuel_s & outer),
  openmc.Cell(name="cladding", fill=clad,
              region=+fuel_s & -clad_s & outer),
  openmc.Cell(name="coolant", fill=cool,
              region=+clad_s & outer)]
Ahex = np.sqrt(3.0)*P**2/2.0
V = np.array([np.pi*rf**2,
              np.pi*(rc**2-rf**2),
              Ahex-np.pi*rc**2])*H
for c, v in zip(cells, V): c.volume = float(v)
```

Geometry sanity checks

$$\sum_r V_r = A_{\text{cell}} H, \qquad A_{\text{sq}} = P^2, \qquad A_{\text{hex}} = \frac{\sqrt{3}}{2} P^2.$$

Use `model.plot()` to inspect the geometry before spending histories. Overlaps, gaps, a missing outer boundary, and a wrong hexagon side length are the most expensive "small" mistakes.

OpenMC unit-cell modeling: run and reduce

How do geometry and materials become defensible physical results?

19 LESSON

Read: Use after defining `cells` and `V` on page 1.

6 Eigenvalue settings

```
settings = openmc.Settings(run_mode="eigenvalue")
settings.batches, settings.inactive = 60, 10
settings.particles, settings.seed = 2000, 19001
settings.source = openmc.IndependentSource(
    space=openmc.stats.Point((0., 0., 0.)),
    angle=openmc.stats.Isotropic(),
    energy=openmc.stats.Watt())
settings.temperature = {"method": "interpolation"}
```

Inactive batches converge the fission source; active batches estimate responses. Increase histories until uncertainty is small relative to the difference you want to interpret. A random seed aids reproducibility; an independent seed is appropriate for a confirmation calculation.

7 Tallies: filter first, then score

```
cell_filter = openmc.CellFilter(cells)

# PWR groups: thermal, intermediate, fast [eV]
edges = [1.0e-5, 1.0, 1.0e5, 2.0e7]
rates = openmc.Tally(name="regional rates")
rates.filters = [cell_filter, openmc.EnergyFilter(edges)]
rates.scores = ["flux", "absorption", "nu-fission"]

# Optional fine spectrum, logarithmic in energy
E = np.geomspace(1.0e-5, 2.0e7, 161)
spec = openmc.Tally(name="regional spectrum")
spec.filters = [cell_filter, openmc.EnergyFilter(E)]
spec.scores = ["flux"]
```

For a one-group SFR comparison, omit the `EnergyFilter` from `rates`. OpenMC reaction-rate scores already contain the spatial and energy integrals. Do not multiply them by volume again.

8 Assemble and run cleanly

```
model = openmc.Model(
    geometry=openmc.Geometry(cells),
    materials=openmc.Materials([fuel, clad, cool]),
    settings=settings,
    tallies=openmc.Tallies([rates, spec]))

with tempfile.TemporaryDirectory(prefix="ne630_") as d:
    sp_path = model.run(cwd=d, output=True)
    with openmc.StatePoint(sp_path) as sp:
        keff = sp.keff
        t = sp.get_tally(name="regional rates")
        flux = t.get_values(scores=["flux"], value="mean").copy()
        absorp = t.get_values(scores=["absorption"],
                              value="mean").copy()
        prod = t.get_values(scores=["nu-fission"],
                            value="mean").copy()
        flux = flux.reshape(len(cells), -1)
        absorp = absorp.reshape(len(cells), -1)
        prod = prod.reshape(len(cells), -1)
print(keff)
```

Copy arrays while the statepoint is open. The temporary directory prevents XML, statepoint, summary, and tally files from cluttering the repository.

9 Normalize only for the question asked

With a cell filter, the flux score Φ_g^r is integrated over cell r. For a regional volume-average flux,

$$\overline{\phi}_g^r = \frac{\Phi_g^r}{V_r}.$$

For a spectrum per unit lethargy, also divide by $\Delta u_g = \ln(E_{g+1}/E_g)$:

```
# after reshaping to [region, energy_bin]
du = np.log(E[1:] / E[:-1])
phi_u = spectrum_flux / V[:, None] / du[None, :]
```

Raw cell-filtered fluxes mainly reflect unequal cell volumes; compare Φ^r/V_r when testing whether regional fluxes are similar.

10 Reduce rates without losing the physics

For each region/group, effective macroscopic data follow from the measured rates:

$$\overline{\Sigma}_{a,g}^r = \frac{A_g^r}{\Phi_g^r}, \qquad \overline{\nu\Sigma}_{f,g}^r = \frac{B_g^r}{\Phi_g^r}.$$

The no-leakage, full-energy rate balance is

$$\frac{B}{A} = \frac{\sum_{g,r} B_g^r}{\sum_{g,r} A_g^r}.$$

For the PWR three-group ordering T, I, F and regions fuel, clad, coolant:

```
A_T = absorp[:, 0].sum()
A_I_f = absorp[0, 1]
B_T, B_F = prod[0, 0], prod[0, 2]
epsilon = (B_T + B_F)/B_T
p       = A_T/(A_T + A_I_f)
f       = absorp[0, 0]/A_T
eta_T   = B_T/absorp[0, 0]
k4 = epsilon*p*f*eta_T
```

The reduced four-factor product omits terms by construction; report those omissions instead of forcing it to equal `keff`.
For the SFR, test the common-flux approximation by comparing measured B/A with

$$k_{\text{common flux}} = \frac{\sum_r V_r \overline{\nu\Sigma}_f^r}{\sum_r V_r \overline{\Sigma}_a^r}.$$

11 Before believing the answer

- Confirm material density, temperature, enrichment *basis*, and thermal scattering treatment.
- Confirm region signs, no overlaps/gaps, and reflective boundaries.
- Check $\sum V_r$, plot the geometry, and keep tally ordering explicit.
- Report $k \pm \sigma_k$; compare changes with Monte Carlo uncertainty.
- Change one input at a time. A unit cell cannot determine leakage, depletion, control worth, thermal limits, or whole-core safety margins.

OpenMC | Minimal experiment pattern

Write one builder `build_model(parameters)` and one reducer `run_case(parameters)`. Sweep pitch for the PWR or enrichment for the SFR, retain every input and uncertainty, bracket a target, then perform one higher-statistics confirmation run with an independent seed.

Companion notebooks: `notebooks/lesson_17.ipynb` (PWR) and `notebooks/lesson_19.ipynb` (SFR). OpenMC references: materials, geometry, settings, and tallies.